

# Sentence-BERT を用いた用途・予算別 PC 構成推薦システム

学籍番号 24026111, 氏名佐藤信晃

2026 年 7 月 25 日

## 1 課題設定

### 1.1 背景と解決する問題

PC を購入する際には, CPU, GPU, メモリ, ストレージ, 電源容量など, 多数の部品を用途に応じて選ぶ必要がある。しかし, 初心者が「WQHD でゲームを快適に遊びたい」などの要求を持っていても, 個々の部品名から適切な構成を判断することは難しい。また, 同じ用途でも「動画制作」と「YouTube 用の映像編集」のように表現が異なるため, 単純な文字列一致だけでは要求に近い構成を検索できない場合がある。

そこで本課題では, 利用者が自然文で入力した用途と予算から, 意味の近い PC 構成を推薦するプロトタイプを実装した。自然文の意味的な近さを扱うため, Sentence-BERT (SBERT) による文章埋め込みを中核技術として使用する。

### 1.2 対象ユーザと要求

対象ユーザは, PC 部品について詳しくない学生, ゲーマー, 動画制作者などである。主な要求を以下に示す。

- 専門用語を使わず, 自然文で用途を入力できる。
- 指定した予算を超えない構成だけを比較できる。
- 推薦された CPU, GPU, RAM などを一覧で確認できる。
- 推薦理由と入力文に対する類似度を確認できる。

### 1.3 想定ユースケース

利用者は Web 画面に「Apex を高フレームレートで遊びたい」と入力し, 予算を 22 万円に設定する。システムは予算内の候補だけを抽出し, 入力文と各候補の登録用途との意味的類似度を計算する。そして, 類似度の高い順に PC 構成, 概算価格, 推薦理由, 予算残額を表示する。例えば, この入力では競技 FPS 向けの構成が上位になることを期待する。

## 2 技術概要

### 2.1 Sentence-BERT

BERT を用いて文の類似度を求める単純な方法では, 文の組を一つずつモデルへ入力する必要があり, 多数の候補との比較には計算量が大きい。Sentence-BERT は Siamese Network 構造を用いて, 各文を固定長ベクトルへ変換し, ベクトル間の距離によって文の意味的類似度を高速に計算できるようにした手法である [1]。

本システムでは, 日本語を含む 50 以上の言語を扱い, 文や短い段落を 384 次元のベクトルへ写像する paraphrase-multilingual-MiniLM-L12-v2 を使用した [2]。PC 用途の登録文と利用者の入力文は, それぞれ同じ埋め込み空間のベクトルへ変換される。

## 2.2 コサイン類似度

入力文の埋め込みを  $q$ 、候補文の埋め込みを  $d$  とすると、コサイン類似度は式 (1) で定義される。

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\|_2 \|d\|_2} \quad (1)$$

値が大きい候補ほど、入力文と意味的に近いと判断する。実装では埋め込み生成時に L2 正規化を行った後、`sentence_transformers.util.cos_sim` を用いて類似度を計算した。Sentence Transformers の公式文書でも、文埋め込み同士の意味的類似度の算出方法としてコサイン類似度が示されている [3]。

## 2.3 推薦用コーパス

推薦用コーパスは CSV 形式で独自に作成した 26 件の PC 構成である。各行は用途文、概算価格、CPU、GPU、RAM、SSD、電源、カテゴリ、推薦理由を持つ。ゲーム、配信、動画編集、3D 制作、画像生成 AI、機械学習、学生向けなど、異なる目的を含めた。

本プロトタイプのは市場価格を厳密に見積もることではなく、文章埋め込みによる意味検索の動作を検証することである。したがって、価格と部品構成は固定した検証用データであり、実店舗の在庫や価格を外部 API から取得していない。

# 3 システムの設計

## 3.1 入出力

入力は用途を表す自然文、予算、表示候補数の三つである。予算は 14 万円から 60 万円、候補数は 1 件、3 件、5 件から指定できる。出力は類似度順の PC 構成であり、各候補についてカテゴリ、登録用途、CPU、GPU、RAM、SSD、電源、概算価格、推薦理由、予算残額を表示する。

## 3.2 処理フロー

システム全体の処理フローを図 1 に示す。

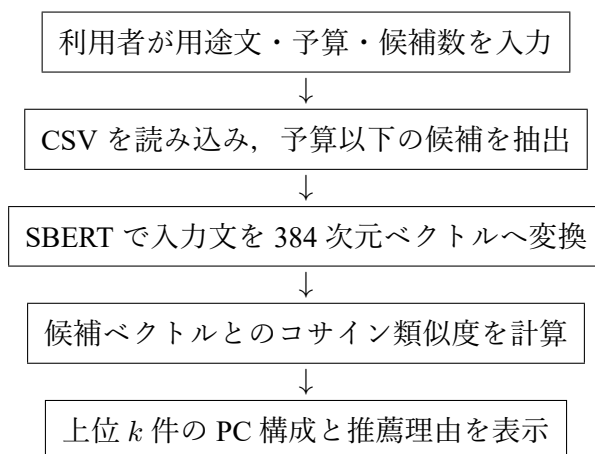


Figure 1: PC 構成推薦システムの処理フロー

## 3.3 モジュール構成

プロトタイプは Python と Streamlit で実装した。主な関数を表 1 に示す。

Table 1: 主な関数と役割

| 関数                                  | 役割                       |
|-------------------------------------|--------------------------|
| <code>load_data</code>              | CSV の読み込みと必須列の検証         |
| <code>load_model</code>             | Sentence-BERT モデルの読み込み   |
| <code>load_corpus_embeddings</code> | 全候補文の事前ベクトル化             |
| <code>recommend</code>              | 予算抽出, 類似度計算, 上位候補の生成     |
| <code>evaluate</code>               | 固定した 10 件による Top- $k$ 評価 |

### 3.4 キャッシュ設計

モデルの読み込みと全文の埋め込み計算を検索のたびに行うと応答が遅くなる。そこで、モデルとコーパス埋め込みには `st.cache_resource` を適用した。CSV 読み込みには `st.cache_data` を適用した。初回検索ではモデル読み込みが必要だが、2 回目以降は入力文だけをベクトル化すればよい。

## 4 プロトタイプの実装

### 4.1 推薦処理

推薦処理では、最初に空文字、予算、候補数を検証する。次に、CSV の全候補から概算価格が予算以下の行番号だけを取得する。入力文を正規化済みベクトルに変換し、対象候補のベクトルとのコサイン類似度を一括計算する。最後に `topk` で上位候補を選び、類似度と予算残額を付加して返す。中核部分を以下に示す。

```
query_embedding = model.encode(
    query,
    convert_to_tensor=True,
    normalize_embeddings=True,
)
eligible_embeddings = corpus_embeddings[eligible_indices]
scores = util.cos_sim(query_embedding,
                      eligible_embeddings)[0]
top = scores.topk(k=min(top_k, len(eligible_indices)))
```

### 4.2 ユーザーインターフェース

画面は「構成を検索」と「性能評価」の二つのタブで構成した。検索タブでは、自然文入力、予算スライダー、候補数選択を一つの画面に配置した。結果は候補ごとの枠に分け、部品名と概算価格を表示する。類似度だけでなく、登録用途と推薦理由も表示することで、利用者が推薦結果を解釈しやすいようにした。

性能評価タブでは、ボタン一つで 10 件を評価し、Top-1 正解率、Top-3 正解率、各入力の期待カテゴリと予測カテゴリを表示する。これにより、推薦結果を目視するだけでなく、同じ条件で繰り返し比較できる。

### 4.3 評価方法

CSV の用途文をそのまま入力すると評価が過度に容易になるため、意味を保った言い換え文を 10 件用意した。各入力には、正解と期待するカテゴリと、そのカテゴリの構成が含まれる予算を設定した。

評価指標は Top-1 正解率と Top-3 正解率である。テスト数を  $N$ 、正解カテゴリを  $y_i$ 、上位  $k$  件のカテゴリ集合を  $P_i^{(k)}$  とすると、Top- $k$  正解率を式 (2) で求める。

$$\text{Accuracy}@k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[y_i \in P_i^{(k)}] \quad (2)$$

## 4.4 評価結果

評価結果を表 2 に示す。

Table 2: 言い換え入力 10 件に対する評価結果

| 指標    | 正解数   | 正解率    |
|-------|-------|--------|
| Top-1 | 10/10 | 100.0% |
| Top-3 | 10/10 | 100.0% |

例えば「WQHD でゲームを快適に遊びたい」、予算 28 万円という入力に対し、1 位は「WQHD ゲーム」で、コサイン類似度は 0.883 であった。2 位は「競技 FPS」で 0.741、3 位は「低予算 e スポーツ」で 0.634 であり、用途の意味に対応した順位となった。

全 10 件で正解したことから、今回用意した用途の範囲では、登録文と完全一致しない入力でも意味的に近い構成を取得できた。一方、評価データは開発者が作成した 10 件のみであり、未知の利用者による評価ではない。したがって、100% という値を一般的な推薦性能として解釈することはできない。

## 5 最終的なシステムの考察

### 5.1 期待される成果

完成形では、利用者が部品名を知らなくても、用途と予算だけで比較の出発点となる PC 構成を得られる。キーワード一致ではなく文章埋め込みを使用するため、「動画編集」と「YouTube 用の映像制作」のような言い換えにも対応できる。また、外部の生成 AI API に依存せず、ローカルで同じ入力に対して再現可能な類似度を計算できる。

### 5.2 問題点

第一に、コーパスが 26 件と小さく、候補に存在しない用途や複合的な要求には弱い。類似度は文章同士の意味の近さを表すが、PC 部品間の互換性や、実際のフレームレートを保証する値ではない。

第二に、価格を固定値としているため、市場価格の変動や在庫を反映できない。また、「静音」「省電力」などを入力しても、候補文の情報だけで順位が決まり、消費電力や騒音の実測値を用いた制約判定は行っていない。

第三に、評価用入力と期待カテゴリを開発者が作成している。同じ人物がコーパスと評価データを作ることによって、表現やカテゴリに偏りが入る。実利用者が自由に入力する場合の誤記、長文、曖昧な要求については追加検証が必要である。

### 5.3 改善案

今後は用途文と構成を増やし、複数人が作成した評価データを訓練・調整用と最終評価用に分離する。価格、消費電力、GPU メモリ、筐体サイズなどは文章類似度とは別の構造化条件として扱う。類似度が低い場合には無理に推薦せず、追加質問や「該当候補なし」を提示する閾値も必要である。

さらに、SBERT による検索結果を第一段階の候補抽出とし、第二段階で部品互換性と定量条件を規則ベースで検証する構成が考えられる。この二段階構成により、文章の柔軟な検索能力と、PC 構成に必要な厳密な制約判定を両立できる。

## 5.4 まとめ

本課題では、多言語 Sentence-BERT を用いて、利用者の用途文と PC 構成の登録用途を 384 次元の文章埋め込みへ変換し、コサイン類似度と予算条件から候補を推薦するプロトタイプを実装した。固定した言い換え入力 10 件では Top-1, Top-3 とともに 100% となり、小規模な対象範囲で意味検索が機能することを確認した。今後はコーパスと独立評価データの拡充、価格情報の更新、部品互換性の検証を追加する必要がある。

## References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992, 2019. <https://arxiv.org/abs/1908.10084>
- [2] Sentence Transformers, “paraphrase-multilingual-MiniLM-L12-v2 Model Card,” <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>, 閲覧日：2026 年 7 月 25 日.
- [3] Sentence Transformers, “Semantic Textual Similarity,” [https://sbert.net/docs/sentence\\_transformer/usage/semantic\\_textual\\_similarity.html](https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html), 閲覧日：2026 年 7 月 25 日.
- [4] Streamlit, “Streamlit documentation,” <https://docs.streamlit.io/>, 閲覧日：2026 年 7 月 25 日.

## A LLM の利用記録

本課題では、LLM を以下の目的で利用した。

- 課題文の要件整理と、既存プロトタイプとの差分確認
- 文字化けおよび Python 構文エラーの修正
- 入力検証, キャッシュ, 性能評価処理の実装支援
- 本文構成と表現の推敲

主な指示は「NLPreport.md の要件に沿って、既存の PC パーツ推薦プロトタイプを完成させること」であった。LLM の出力はそのまま採用せず、Python の構文検査、依存ライブラリのインポート、Sentence-BERT による実検索、10 件の一括評価を実行して確認した。実際の対話全文および詳細なプロンプトは、提出時に別の付帯 PDF へ収録する。

## B ソースコード

ソースコード全文は付帯資料とし、本節では主要ファイルのみを示す。

- app.py : Streamlit 画面, 推薦処理, 評価処理
- pc\_builds.csv : 26 件の PC 構成コーパス
- requirements.txt : 実行に必要なライブラリ

実行コマンドは以下である。

```
.\venv\Scripts\python.exe -m streamlit run app.py \
--server.fileWatcherType none
```

## C 評価結果の生データ

| No. | 期待カテゴリ   | 1 位カテゴリ  | Top-3 |
|-----|----------|----------|-------|
| 1   | 競技 FPS   | 競技 FPS   | 正解    |
| 2   | ゲーム+動画編集 | ゲーム+動画編集 | 正解    |
| 3   | 生成 AI    | 生成 AI    | 正解    |
| 4   | WQHD ゲーム | WQHD ゲーム | 正解    |
| 5   | 4K ゲーム   | 4K ゲーム   | 正解    |
| 6   | ゲーム配信    | ゲーム配信    | 正解    |
| 7   | 3D 制作    | 3D 制作    | 正解    |
| 8   | 学生向け     | 学生向け     | 正解    |
| 9   | シミュレーション | シミュレーション | 正解    |
| 10  | VR ゲーム   | VR ゲーム   | 正解    |